Proyecto Final
Entrega 1: Análisis de Requerimientos y Diseño Conceptual del Sistema

1. Objetivo de la entrega

El proyecto del semestre consiste en que cada equipo diseñe un sistema de software propuesto por el propio equipo. Esta es la primera entrega del proyecto. En ella, el equipo especifica qué debe hacer el sistema y, a partir de esa especificación, identifica las clases que lo forman, con sus atributos, sus métodos y sus relaciones.

Los ejemplos de este documento corresponden a un sistema de citas para un consultorio dental. Se incluyen solo como referencia.

2. Restricciones para la propuesta

El sistema propuesto debe cumplir, como mínimo, con las siguientes condiciones:

- Tener **2 actores humanos** con tareas distintas.
- Comunicarse con **1 sistema externo** (pasarela de pago, servicio de correo o SMS, mapas, etc.).
- Contar con al menos un elemento que **cambie de estado** (por ejemplo, una cita que pasa de *agendada* a *atendida* o *cancelada*).
- Tener **entre 10 y 15 requerimientos funcionales**.

No se aceptan sistemas demasiado grandes, como una red social, un ERP o un marketplace, porque no pueden diseñarse en un semestre.

3. Paso 1: Especificación de Requerimientos de Software (SRS)

El SRS es el documento que describe **qué** debe hacer el sistema, sin indicar **cómo** se programará. El equipo debe elaborar el SRS del sistema propuesto con las siete secciones que se describen a continuación, en el mismo orden y con la misma numeración. Si una sección no aplica al sistema, se escribe “No aplica” y se explica el motivo en una línea.

Sección 1. Alcance y glosario

La sección inicia con un párrafo que explica qué problema resuelve el sistema y a quién beneficia. Después, el alcance se delimita con dos listas:

- **Dentro del alcance:** lo que el sistema sí hará.
- **Fuera del alcance:** al menos 3 funciones que el sistema no hará, aunque alguien pudiera esperarlas (por ejemplo, “El sistema no cobra los tratamientos; solo agenda citas”).

La sección termina con un glosario de al menos 6 términos propios del negocio, cada uno con su definición. Cuando dos términos sean parecidos, el glosario debe explicar en qué se diferencian (por ejemplo, *cita* y *consulta*). Cada término se usa siempre con el mismo significado en todo el documento.

Sección 2. Contexto y actores

La sección inicia con un párrafo que describe la situación actual:

- Si el proceso ya existe, se explica cómo se realiza hoy y qué cambiará con el sistema propuesto.
- Si el sistema es completamente nuevo, se explica qué necesidad no se atiende actualmente y a quién afecta.

Después, el equipo debe diseñar el **diagrama de contexto** del sistema propuesto, con las siguientes características:

- El sistema se representa como una sola caja al centro.
- Todos los actores (personas y sistemas externos) se colocan alrededor del sistema.
- Cada actor se une con el sistema mediante una flecha etiquetada con la información que intercambian (por ejemplo, Paciente → Sistema: “solicitud de cita”).
- El diagrama no muestra el interior del sistema (pantallas, módulos o base de datos).

El diagrama se complementa con una **tabla de actores** que indica, para cada actor, si es una persona o un sistema externo y qué hace en el sistema. Para cada sistema externo se especifica, además, qué información le envía el sistema, qué información recibe y qué ocurre si no responde (por ejemplo, "si el servicio de SMS falla, la cita se guarda y el recordatorio se reintenta cada hora").

Sección 3. Restricciones

Una restricción es una limitación que el equipo **no eligió**, sino que le es impuesta. Puede referirse a:

- La plataforma: "debe funcionar en el navegador, tanto en computadora como en celular".
- Servicios obligatorios: "debe usar el servicio de SMS que ya contrató el consultorio".
- Leyes: "debe cumplir la Ley Federal de Protección de Datos Personales".

Si el sistema no tiene restricciones, se escribe "No aplica".

Sección 4. Supuestos y dependencias

Un **supuesto** es algo que el equipo da por cierto sin haberlo confirmado. Una **dependencia** es un elemento externo sin el cual el sistema no puede funcionar. Cada vez que el equipo desconozca algo del negocio, debe registrarlo como supuesto e indicar qué cambiaría si resulta falso, en lugar de decidirlo sin dejar constancia.

La sección debe contener al menos **6 supuestos y 2 dependencias**, registrados en una tabla con el siguiente formato:

ID	Tipo	Pregunta sin respuesta	Lo que se asume	Qué cambia si es falso
SUP-01	Supuesto	¿Cuánto dura una cita?	Todas las citas duran 30 min	Habría que manejar duraciones distintas en la clase Cita
DEP-01	Dependencia	¿Qué pasa si el SMS falla?	El servicio de SMS funciona el 99 % del tiempo	No llegarían recordatorios; habría que agregar avisos por correo

Los supuestos obvios, como "el usuario tiene internet", no cuentan para el mínimo.

Sección 5. Requerimientos funcionales (RF)

Un requerimiento funcional es una acción que el sistema debe realizar. La sección debe contener entre 10 y 15 RF, registrados en una tabla con el siguiente formato:

ID	Requerimiento	Prioridad	RN	Criterio de aceptación
RF-05	El paciente puede cancelar una cita agendada	Must	RN-01	**Caso correcto:** faltan más de 24 h → la cita queda *cancelada*. **Caso de error:** faltan menos de 24 h → la cita no se cancela y se muestra un mensaje

Cada columna se llena de la siguiente manera:

- **ID:** RF-01, RF-02, etc.
- **Requerimiento:** una sola acción por RF, redactada con "El sistema debe…" o "El [actor] puede…".
- **Prioridad:** *Must* (indispensable), *Should* (importante) o *Could* (deseable si hay tiempo). Como máximo, el 60 % de los RF puede tener prioridad *Must*.
- **RN:** número de las reglas de negocio (Sección 6) que aplican al requerimiento, o "—" si no aplica ninguna.
- **Criterio de aceptación:** un caso correcto y al menos un caso de error.

Sección 6. Reglas de negocio (RN)

Una regla de negocio es una política que el sistema debe respetar. La sección debe contener al menos 4 reglas, numeradas (RN-01, RN-02, etc.) y redactadas en una sola oración cada una.

El RF describe la **acción** que realiza el sistema; la RN describe la **condición** que esa acción debe respetar. Por esta razón, la regla se escribe solo en esta sección, y en la tabla de RF únicamente se cita su número.

Sección 7. Requerimientos no funcionales (RNF)

Un requerimiento no funcional describe una característica de calidad del sistema. La sección debe contener al menos 3 RNF, por ejemplo, uno de desempeño, uno de seguridad y uno de disponibilidad. Cada RNF debe incluir una medida que permita comprobarlo. Ejemplo: "Los horarios disponibles se muestran en 2 segundos o menos con 100 usuarios simultáneos".

En todo el documento deben evitarse palabras que no se pueden medir, como *rápido*, *fácil*, *intuitivo*, *seguro*, *24/7* o *adecuado*. En su lugar se escriben valores concretos: "en 2 s o menos", "disponible el 99.5 % del mes", "las contraseñas se almacenan cifradas".

4. Paso 2: Diseño conceptual preliminar

En este paso, el equipo transforma los requerimientos en clases. Cada clase, atributo, método y relación debe justificarse con un RF, una RN o un término del glosario del SRS.

4.1 Identificación de clases

El equipo subraya los sustantivos que aparecen en los RF, las RN y el glosario (por ejemplo: *paciente, cita, dentista, horario, recordatorio, fecha, sistema*). Después, analiza cada sustantivo con las preguntas de la siguiente tabla para decidir si es una clase, un atributo o si debe descartarse:

Pregunta	Si la respuesta es sí…	Ejemplo
¿Guarda información propia y puede haber varios de ellos?	Es una **clase**	Cita, Paciente
¿Es solo un dato de otra cosa?	Es un **atributo**	*fecha* es atributo de Cita
¿Es el sistema mismo o una pantalla?	Se **descarta**	"sistema", "menú"
¿Es sinónimo de otra clase?	Se **une** con esa clase	"cliente" = Paciente
¿Es un sistema externo?	Se **descarta**, porque se trata como caja negra	Servicio de SMS

El resultado es una lista de al menos 5 clases.

4.2 Tarjetas CRC de las clases principales

Antes de cerrar la lista de clases, el equipo elabora una tarjeta CRC (*Clase – Responsabilidades – Colaboradores*) para cada una de las **3 clases con más comportamiento**, es decir, las que cambian de estado o aplican reglas de negocio. Las tarjetas sirven para definir qué sabe, qué hace cada clase y con qué otras clases necesita trabajar.

Clase	Responsabilidades (qué sabe y qué hace)	Colaboradores
Cita	Conoce su fecha, hora y estado · Se agenda en un horario libre (RF-02) · Se cancela si faltan más de 24 h (RF-05, RN-01) · Programa su recordatorio (RF-07)	Paciente, Dentista, Recordatorio

Las tarjetas deben cumplir con lo siguiente:

- Cada responsabilidad indica el RF o la RN de donde proviene.
- Cada tarjeta tiene como máximo 6 responsabilidades. Si una clase necesita más, probablemente hace demasiado y debe dividirse.
- Si al elaborar una tarjeta aparece una clase que no estaba en la lista (por ejemplo, *Recordatorio*), se agrega a la lista.

4.3 Atributos y métodos

Cada clase debe tener al menos 3 atributos, cada uno con su nombre y su tipo.

Los métodos de cada clase son únicamente los necesarios para cumplir los requerimientos funcionales. Las responsabilidades de "hacer" de las tarjetas CRC son el punto de partida para definirlos. Consideraciones para definir los métodos deben cumplir con lo siguiente:

- No se incluyen constructores, getters ni setters, porque se dan por hechos.}
- Cada método se justifica con un RF o una RN.

Los atributos y métodos se registran en una tabla con el siguiente formato:

Clase	Atributos	Métodos	Justificación
Cita	fecha: Date · hora: Time · estado: EstadoCita	confirmar() · cancelar() · marcarAtendida()	RF-03, RF-05, RF-06, RN-01
Paciente	nombre: String · telefono: String · correo: String	—	RF-01, glosario

4.4 Relaciones

El equipo identifica cómo se relacionan las clases, usando los tipos de relación vistos en clase: asociación, agregación, composición y herencia. Los colaboradores de las tarjetas CRC indican relaciones probables.

Cada relación se registra en una tabla que indica su tipo, su **multiplicidad** (cuántos objetos participan de cada lado: 1, 0..1, 0..*, 1..*) y su justificación:

Clase A	Relación	Clase B	Multiplicidad	Justificación
Paciente	Asociación	Cita	1 paciente — 0..* citas	RF-02, RN-02

4.5 Diagrama de clases

El equipo diseñará el diagrama de clases UML del sistema con todas las clases, atributos, métodos y relaciones definidos en los apartados 4.1 a 4.4. El diagrama debe cumplir **todas las características de un diagrama de clases vistas en clase**: visibilidad, nombre y tipo de los atributos, y nombre, parámetros (con su tipo) y tipo de retorno de los métodos. El diagrama se elabora con una herramienta digital (draw.io, Lucidchart, PlantUML o similar) y debe ser legible.

5. Formato de entrega

- **Archivos:** se entregan dos archivos:
  1. El documento en PDF, con el nombre `Equipo##_Entrega1_NombreDelSistema.pdf`.
  2. El archivo fuente del diagrama de clases, en el formato de la herramienta utilizada (por ejemplo, `.drawio` o `.puml`), con el nombre `Equipo##_Entrega1_DiagramaClases`
- **Extensión:** no hay un número fijo de páginas. Incluyan la información necesaria para comprender cada sección y eviten contenido que no aporte a su propósito. El documento debe ser claro y suficiente para que una persona ajena al equipo pueda comprender el sistema y su diseño sin explicaciones adicionales.
- **Portada:** nombre del sistema, integrantes del equipo y fecha.
- **Lugar de entrega:** Canvas, en la actividad correspondiente.
- **Fecha límite:** la establecida en canvas